

CultiTech

SEMBRANDO DATOS.

COSECHANDO DECISIONES.

Sistema de Gestión y Control de Campos
de Producción Agrícola.

Autores:

David Lajas Ramos

Ane Arana Alonso

Irina Balica

Álvaro Guerra Cabello

Juan Carlos Díaz Pérez

Patricia Ballesteros Alonso

Sonia Nseng Mikue

Zahir Rivera Chacón

Sergio García Méndez

Julio González Trenard

Ainhize Uriarte Bandera

Índice

1. Introducción y contexto del reto

- 1.1. Contexto actual del sector frutícola y origen del proyecto
- 1.2. Problema de negocio y fricciones identificadas
- 1.3. Objetivo del proyecto y alcance del MVP

2. Diseño global de la solución

- 2.1. Enfoque general y visión de producto
- 2.2. Actores del sistema y casos de uso principales
- 2.3. Requisitos funcionales del MVP
- 2.4. Requisitos no funcionales y criterios de diseño

3. Arquitectura tecnológica de la plataforma

- 3.1. Principios de diseño de la arquitectura
- 3.2. Componentes del sistema y responsabilidades
- 3.3. Tecnologías utilizadas y justificación de la elección
- 3.4. Flujo de datos y funcionamiento general del sistema

4. Catálogo de APIs y modelo de integración

- 4.1. Enfoque estratégico del catálogo de APIs
- 4.2. Organización funcional de las APIs
 - 4.2.1. APIs climáticas y meteorológicas
 - 4.2.2. APIs de riesgos y predicción agrícola
 - 4.2.3. APIs de visión artificial
 - 4.2.4. APIs de suelo y productividad
 - 4.2.5. API de asistencia inteligente
- 4.3. Relación entre APIs y perfiles de usuario

5. Gestión de datos e inteligencia en la plataforma

- 5.1. Datos al servicio del producto
- 5.2. Identificación de necesidades y fuentes de información
- 5.3. Preparación y estructuración de datos para la web
- 5.4. Desarrollo de servicios de inteligencia
- 5.5. Validación funcional de los servicios de datos

6. Inteligencia aplicada y resultados del MVP

- 6.1. La inteligencia como capa transversal del producto
- 6.2. Decisiones apoyadas por la plataforma
- 6.3. Uso de datos climáticos y visuales

- 6.4. Resultados obtenidos en el MVP
- 6.5. Evolución prevista de la inteligencia del sistema

7. Validación, limitaciones y evolución del producto

- 7.1. Proceso de validación del MVP
- 7.2. Límites actuales del MVP
- 7.3. Líneas de evolución y mejora del producto

8. Conclusiones

Anexo 1

Documentación técnica APIs Cultitech

Anexo 2

Referencias técnicas y recursos externos

1. Introducción y contexto del reto

1.1 Contexto y origen del proyecto

La producción frutícola se desarrolla hoy en un entorno cada vez más inestable e impredecible. El cambio climático, con eventos extremos más frecuentes e intensos, ha pasado de ser un riesgo puntual a convertirse en una variable estructural que condiciona la producción, la calidad del producto y la viabilidad económica de las explotaciones.

A este contexto se suma una cadena de valor compleja, en la que productores, técnicos, asesores y managers se encuentran distribuidos geográficamente y presentan necesidades muy distintas. Aunque existe una gran cantidad de datos disponibles -climatología, imágenes de campo, observaciones técnicas o registros productivos- en muchos casos esta información se encuentra fragmentada, no se comparte de forma eficiente y resulta difícil transformarla en decisiones concretas.

Este proyecto nace como respuesta a una necesidad real planteada por el propio sector frutícola, utilizando Chile como caso piloto inicial. El contexto chileno resulta especialmente representativo debido a su alta variabilidad climática, a una producción orientada a mercados internacionales y a una fuerte exigencia en términos de calidad, trazabilidad y cumplimiento de compromisos comerciales.

Desde el inicio, la solución se plantea con una clara mentalidad de producto, y no como un desarrollo puntual. El objetivo es crear una plataforma escalable, capaz de adaptarse a otros países y contextos agrícolas, manteniendo una base tecnológica común.

El proyecto ha sido desarrollado por un equipo multidisciplinar de Full Stack y Data Science, abordando el reto de forma conjunta desde la tecnología, el dato y la realidad del negocio agrícola.

1.2 Problema de negocio

El análisis del problema permitió identificar tres grandes fricciones que afectan de forma directa al día a día del sector frutícola.

En primer lugar, la toma de decisiones reactiva. Muchos eventos críticos -como heladas, olas de calor, aparición de plagas o cambios en el estado del cultivo- se gestionan cuando el impacto ya se ha producido, lo que reduce el margen de actuación y aumenta las pérdidas.

En segundo lugar, la dispersión de la información. Los datos climáticos, las imágenes de campo y los registros técnicos suelen encontrarse repartidos en distintas herramientas o incluso fuera de sistemas digitales, sin integración ni trazabilidad temporal o geográfica.

Por último, existe una falta de alineación entre los distintos actores de la cadena. Productores, asesores y managers trabajan con información parcial y con criterios distintos, lo que dificulta la coordinación, especialmente en situaciones de crisis donde el tiempo y la calidad de la información son factores clave.

Estas fricciones impactan directamente en la productividad, los costes operativos, la calidad final del producto y la capacidad de planificación y negociación en mercados cada vez más exigentes.

1.3 Objetivo del proyecto

El objetivo del proyecto es desarrollar un MVP (Producto Mínimo Viable) de plataforma web para el control de campos de producción agrícola, que combine de forma coherente:

- un sistema Full Stack (frontend y backend), responsable de la interacción con los usuarios, la gestión de flujos y la seguridad;
- y un módulo de Data Science e Inteligencia Artificial, encargado de analizar datos, anticipar riesgos y apoyar el diagnóstico técnico.

La plataforma está diseñada para apoyar la toma de decisiones de los distintos perfiles de la cadena frutícola -productores, asesores agrícolas y managers- mediante:

- la integración de datos climáticos, cada vez más relevantes en un contexto de cambio climático;
- el uso de imágenes y evidencias visuales del estado real del cultivo;
- y la adaptación de la información al rol de cada usuario, facilitando su interpretación y uso práctico.

En todo momento, la solución se plantea como una herramienta de apoyo, y no como un sustituto del criterio humano, especialmente en un sector donde la experiencia y el conocimiento del terreno siguen siendo fundamentales.

Diagrama 1 — Modelo conceptual del sistema CultiTech

Sector frutícola



2. Requerimientos y diseño global de la solución

2.1 Enfoque general de la solución

La solución se plantea como una plataforma web modular orientada al control y seguimiento de campos de producción agrícola, con un objetivo claro: centralizar la información relevante del cultivo y convertirla en apoyo real a la toma de decisiones.

Desde el inicio, el diseño se ha realizado pensando en un producto usable en el día a día, no en una herramienta experimental. Por este motivo, se ha apostado por una arquitectura que permita crecer de forma progresiva, incorporar nuevas funcionalidades y adaptarse a distintos contextos agrícolas y geográficos sin necesidad de rediseñar el sistema completo.

La plataforma se estructura en tres grandes capas, claramente diferenciadas pero estrechamente coordinadas, reflejando el trabajo conjunto de las dos disciplinas del proyecto:

- Frontend web, centrado en la experiencia de usuario, la visualización de información compleja y el acceso sencillo desde distintos dispositivos.
- Backend, encargado de la lógica de negocio, la seguridad, la gestión de usuarios y la orquestación de servicios.
- Servicios de Data Science & AI, responsables del análisis de datos, la predicción de riesgos, el análisis visual y la asistencia inteligente.

Esta separación permite que cada componente evolucione de forma independiente, manteniendo estabilidad y coherencia en el conjunto.

2.2 Actores y casos de uso principales

El diseño funcional de la plataforma parte de una premisa clave: no todos los usuarios necesitan la misma información ni la interpretan de la misma manera. Por ello, el sistema se adapta al rol de cada actor dentro de la cadena frutícola.

El productor necesita una visión clara y directa de lo que ocurre en su campo: condiciones climáticas, alertas tempranas, estado y registro del cultivo y registro de incidencias. La plataforma le permite anticiparse a problemas y documentar la realidad del campo con evidencias objetivas.

El asesor agrícola requiere un mayor nivel de detalle. Accede a información integrada, análisis y evidencias visuales que le permiten emitir recomendaciones fundamentadas y dar soporte técnico a distancia, especialmente en situaciones críticas.

El manager, por su parte, necesita información fiable y trazable sobre el estado productivo y sanitario del cultivo. La plataforma le aporta una base objetiva para la planificación logística, la negociación comercial y la gestión de riesgos en la cadena de suministro.

La plataforma actúa así como un punto de encuentro basado en datos, alineando a perfiles con intereses distintos pero interdependientes.

2.3 Requisitos funcionales del MVP

El MVP se ha diseñado priorizando aquellas funcionalidades que aportan mayor valor inmediato, especialmente en un contexto marcado por el cambio climático y la necesidad de reacción rápida ante incidencias.

Entre las capacidades funcionales principales se incluyen:

- La integración de datos climáticos, tanto actuales como históricos, como base para el análisis de riesgos.
- La generación de alertas y predicciones ante eventos climáticos adversos y riesgos fitosanitarios.
- El análisis visual del cultivo a partir de imágenes, permitiendo detectar plagas, identificar cultivos y registrar el estado real del campo.
- El acceso a toda la información mediante APIs REST, facilitando su consumo desde la plataforma web y su futura integración con sistemas externos.
- Una trazabilidad básica de datos, predicciones e imágenes asociadas a parcela y fecha, fundamental para la toma de decisiones y la comunicación entre actores.

Estas funcionalidades definen el alcance del MVP y establecen una base sólida para la evolución futura del producto.

2.4 Requisitos no funcionales

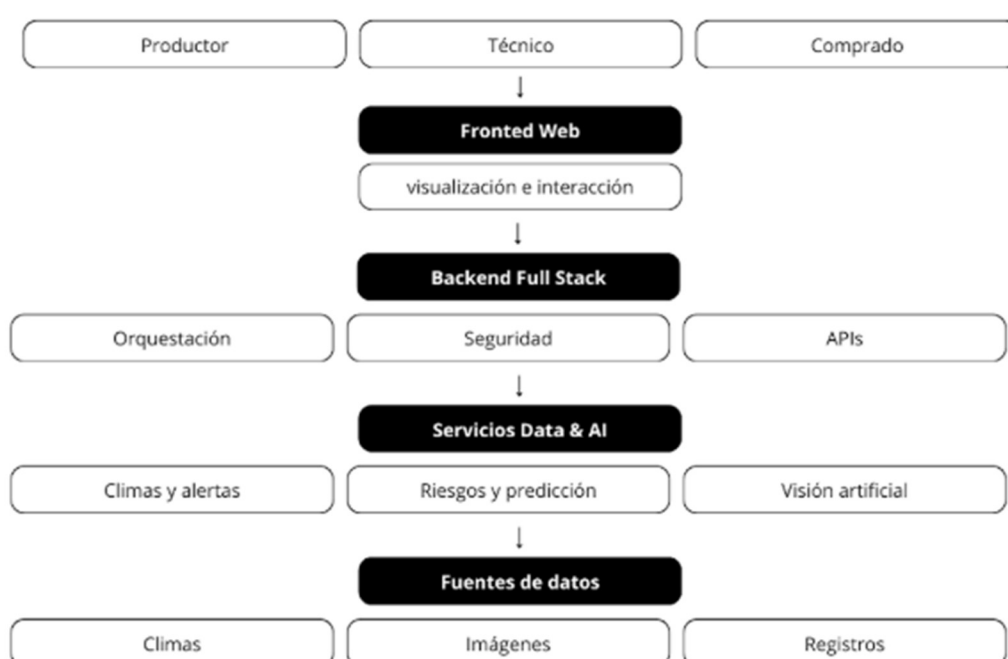
Además de las funcionalidades, el diseño de la plataforma incorpora una serie de requisitos no funcionales clave para garantizar su viabilidad como producto:

- Modularidad y desacoplamiento, evitando dependencias rígidas entre frontend, backend y servicios de datos.
- Escalabilidad, permitiendo incorporar nuevos cultivos, plagas, países y escenarios productivos.
- Interoperabilidad, basada en APIs, para facilitar integraciones con terceros.
- Orientación a producto, evitando soluciones específicas para un único cliente o contexto.

Estos requisitos aseguran que el MVP no sea un desarrollo puntual, sino el primer paso hacia una plataforma comercial sostenible.

Diagrama 2 — Diseño global del sistema (visión funcional)

Diagrama 2 — Diseño global del sistema (visión funcional)



3. Arquitectura tecnológica

3.1 Enfoque general de la arquitectura

La arquitectura del sistema se ha diseñado con una idea muy clara desde el principio: crear una base tecnológica sólida, pero flexible, que permita evolucionar el producto sin tener que rehacerlo cada vez que se añade una nueva funcionalidad.

No se trata solo de que “funcione”, sino de que sea mantenible, escalable y comprensible tanto para el equipo técnico como para terceros. Por eso se ha optado por una arquitectura modular, en la que cada parte del sistema tiene una responsabilidad bien definida.

Este enfoque permite que el trabajo del equipo Full Stack y el de Data Science estén claramente separados, pero perfectamente coordinados. Cada disciplina puede avanzar de forma independiente, sin bloquear a la otra, y al mismo tiempo integrarse de forma limpia dentro de la plataforma.

3.2 Estructura del sistema y responsabilidades

A nivel práctico, el sistema se organiza en cuatro bloques principales, que reflejan cómo fluye la información desde el usuario hasta los modelos y de vuelta a la plataforma:

- Frontend web

Es el punto de entrada de los usuarios. Aquí se muestran los datos, las alertas, los resultados de análisis y las evidencias visuales de forma clara y comprensible, adaptadas al perfil de cada usuario.

- Backend

Actúa como el “cerebro” de la aplicación. Gestiona usuarios, permisos, flujos de información y decide qué servicios se activan en cada caso. Además, es el encargado de comunicarse con las APIs y garantizar la seguridad y coherencia de los datos.

- Servicios de Data Science & AI

Aquí se concentra la inteligencia del sistema. Incluye los modelos de predicción climática, los análisis de riesgo, los servicios de visión artificial y los agentes de asistencia inteligente. Estos servicios no están acoplados a la interfaz, sino que se exponen como APIs reutilizables.

- Datos y servicios externos

Incluye tanto las bases de datos propias como las fuentes externas, especialmente los servicios meteorológicos, que aportan información clave en un contexto de cambio climático.

Esta separación permite entender fácilmente qué hace cada parte del sistema y facilita su evolución a futuro.

3.3 Tecnologías utilizadas y criterios de elección

Las tecnologías utilizadas en el proyecto se han seleccionado con base en tres criterios clave: herramientas maduras y contrastadas, facilidad de integración entre equipos, y compatibilidad con la visión de un MVP escalable.

Frontend: React + Vite

El frontend de la plataforma ha sido desarrollado utilizando React, un framework ampliamente reconocido por su capacidad de crear interfaces modulares, rápidas y escalables. La elección de Vite como herramienta de desarrollo proporciona un entorno de construcción multiprocesado optimizado, lo que hace que el proceso de desarrollo sea rápido y eficiente. Estas tecnologías garantizan que el sistema proporcione una experiencia fluida incluso en dispositivos con recursos limitados.

Backend: Node.js + Express

Para el backend se utilizó Node.js, junto con el framework Express para estructurar las APIs REST que soportan la lógica de negocio. Esta elección permitió construir un backend ligero, altamente personalizable y compatible con múltiples servicios externos. Node.js destacó por su capacidad de manejar eventos concurrenciosos y facilitar la rápida iteración, mientras que Express proporcionó una estructura modular para organizar las rutas, middlewares y controladores.

Módulos de Data Science & AI

En lo relacionado con datos, los servicios de Data Science y AI fueron diseñados con Python, utilizando Flask para la exposición de APIs y conectándolos de forma sencilla con el backend principal. Estos módulos manejan tareas analíticas específicas como predicción climática, análisis basado en Machine Learning y detección de imágenes a partir de modelos entrenados.

Persistencia: Bases de Datos y Archivos

El sistema combina bases de datos relacionales para la gestión estructurada de datos (usuarios, cultivos, y predicciones) y almacenamiento de archivos para elementos como imágenes, videos y modelos entrenados. Esta arquitectura garantiza que tanto los servicios de Data Science como las interacciones de usuarios tengan un flujo de información centralizado y trazable.

En conjunto, estas herramientas no solo fueron seleccionadas por sus funcionalidades específicas, sino por su capacidad de coexistir e integrarse, asegurando que el equipo Full Stack y de Data Science trabajaran como un solo equipo en la implementación del MVP.

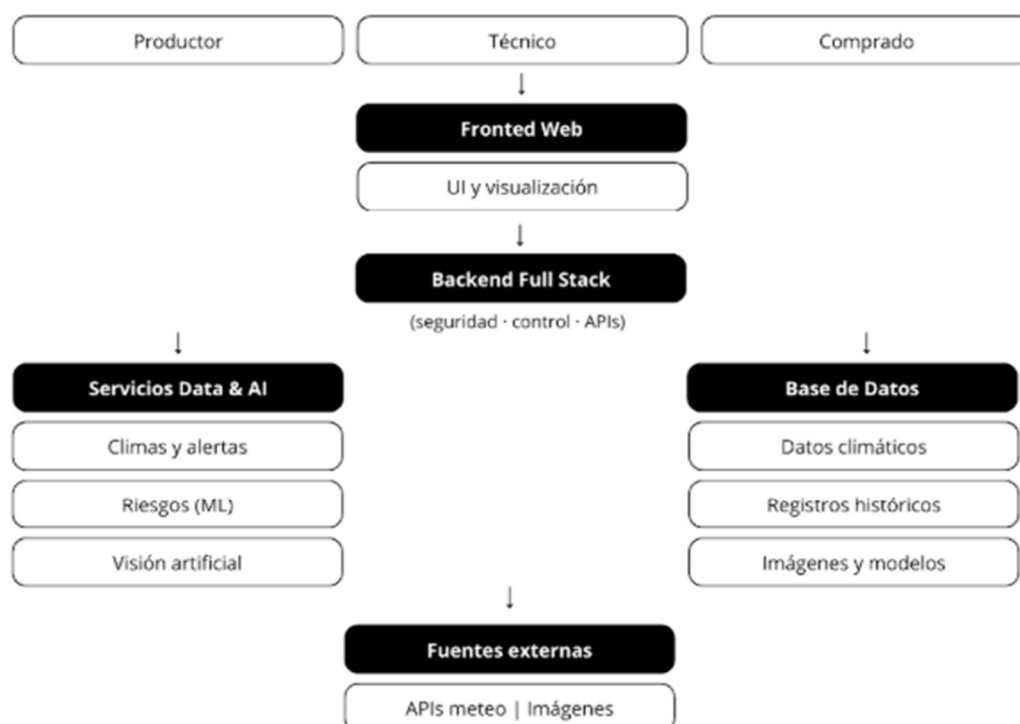
3.4 Flujo de datos y funcionamiento del sistema

El funcionamiento del sistema sigue un flujo sencillo y bien definido. El usuario interactúa con la plataforma a través del frontend, el backend valida la petición y decide qué servicios deben activarse, y los módulos de Data Science procesan la información y devuelven los resultados.

Estos resultados se transforman en información comprensible y se presentan de nuevo al usuario, cerrando el ciclo. Todo este proceso se realiza mediante APIs REST, utilizando formatos estándar, lo que facilita tanto el mantenimiento como la integración con sistemas externos.

Este enfoque garantiza que la plataforma pueda crecer, adaptarse a nuevos países o incorporar nuevos modelos sin necesidad de rediseñar su arquitectura.

Diagrama 3 — Arquitectura tecnológica y flujo de información



4. Catálogo de APIs del sistema

4.1 Enfoque general del catálogo de APIs

El diseño del catálogo de APIs no responde únicamente a una necesidad técnica, sino a una decisión estratégica de producto. Desde el inicio, se optó por una arquitectura basada en APIs para asegurar que la plataforma pudiera crecer, adaptarse y reutilizarse en distintos contextos, clientes y países.

Las APIs actúan como el punto de unión entre el trabajo del equipo Full Stack y el de Data Science. Por un lado, permiten que la plataforma web consuma datos y resultados de forma estructurada; por otro, facilitan que los modelos y servicios de análisis evolucionen de manera independiente, sin afectar a la experiencia del usuario.

El objetivo de este catálogo no es listar endpoints de forma exhaustiva -eso se documenta en un documento técnico específico-, sino mostrar cómo se estructura el sistema, qué tipo de servicios ofrece y cómo estos servicios dan respuesta a las necesidades reales de los distintos perfiles de usuario.

4.2 Organización funcional de las APIs

Las APIs se han organizado en bloques funcionales, alineados con los principales casos de uso del producto. Esta organización facilita tanto su comprensión como su mantenimiento y futura ampliación.

APIs climáticas y meteorológicas

Estas APIs proporcionan acceso a datos climáticos actuales e históricos, que constituyen la base informativa del sistema. En un contexto de cambio climático, este tipo de información es crítica para anticipar riesgos y entender la evolución de las condiciones del cultivo.

APIs de riesgos y predicción agrícola

A partir de los datos climáticos, este bloque transforma la información bruta en alertas y predicciones accionables. Aquí se integran modelos de Machine Learning y reglas agronómicas que permiten detectar eventos adversos y ventanas de riesgo, apoyando decisiones preventivas.

APIs de visión artificial

Este conjunto de APIs permite analizar imágenes y vídeos del campo para obtener evidencias visuales objetivas. Su función principal es apoyar el diagnóstico técnico mediante la detección de plagas, la identificación de cultivos y el registro del estado real del campo, reduciendo la dependencia de observaciones subjetivas.

APIs de suelo y productividad

Estas APIs agrupan información relacionada con el suelo y la productividad del cultivo. Aunque algunas funcionalidades se encuentran en fase inicial, el diseño del sistema ya contempla su integración como parte clave de la toma de decisiones agronómicas.

API de asistencia inteligente

Este servicio actúa como una capa de interpretación entre los resultados técnicos del sistema y los usuarios. Mediante un agente conversacional basado en modelos de lenguaje, la plataforma puede explicar resultados, contextualizar alertas y adaptar el discurso al perfil del usuario, sin sustituir el criterio humano.

4.3 Relación entre APIs y perfiles de usuario

Uno de los aspectos clave del diseño es que las mismas APIs pueden servir a distintos usuarios, pero la forma en que se presentan los resultados varía según el rol.

El productor accede principalmente a información clara y directa: alertas, estado del cultivo y evidencias visuales.

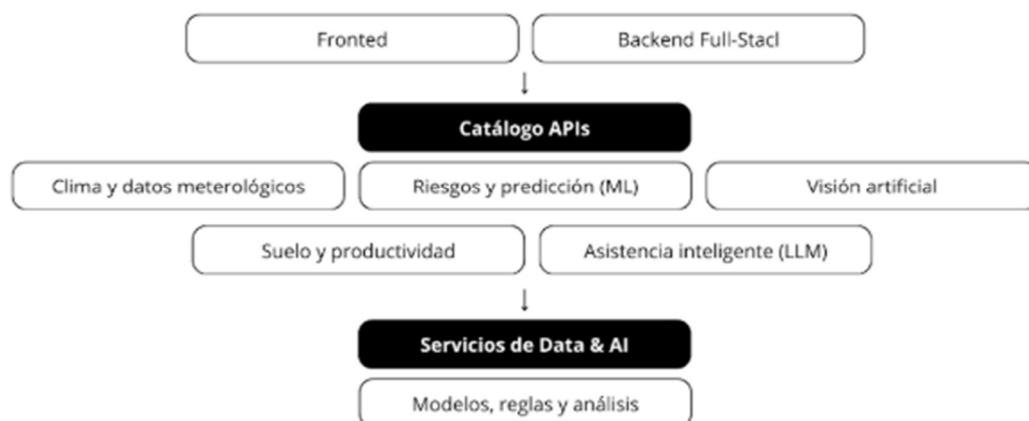
El asesor utiliza las APIs para profundizar en el análisis, revisar históricos y emitir recomendaciones fundamentadas.

El manager obtiene una visión objetiva del estado productivo y sanitario, útil para la planificación logística y la negociación.

Este enfoque garantiza coherencia entre la necesidad de negocio, el diseño funcional y la implementación técnica.

Diagrama 4 — Mapa funcional del catálogo de APIs

Diagrama 4 — Mapa funcional del catálogo de APIs



5. Trabajo con datos e inteligencia dentro de la plataforma

5.1 Enfoque general: datos al servicio del producto

El trabajo con datos en este proyecto no se ha planteado como un proceso aislado ni como un ejercicio puramente analítico. Desde el inicio, los datos y la inteligencia se han trabajado como una parte más del producto, siempre condicionados por una pregunta clave: ¿cómo va a utilizar esta información cada tipo de usuario dentro de la plataforma?

Este enfoque ha marcado la forma de trabajar del equipo. Las decisiones relacionadas con los datos, los análisis y los servicios de inteligencia se han tomado de manera coordinada entre Data Science y Full Stack, teniendo en cuenta tanto la viabilidad técnica como la experiencia de usuario y los flujos reales de uso.

El objetivo no era generar análisis complejos, sino ofrecer información comprensible, útil y accionable dentro de una aplicación web real.

5.2 Identificación de necesidades y fuentes de información

El punto de partida del trabajo con datos fue siempre el problema del negocio, no la disponibilidad de tecnología. A partir de los escenarios definidos -como la gestión de una helada, la detección de una plaga o el seguimiento del estado del cultivo- se identificaron las necesidades de información de cada perfil: productor, técnico y manager.

Para dar respuesta a estas necesidades, se trabajó principalmente con:

- Datos climáticos, tanto actuales como históricos, fundamentales para entender el contexto y anticipar riesgos en un entorno marcado por el cambio climático.
- Imágenes de campo, que permiten documentar el estado real del cultivo de forma objetiva y compatible.
- Registros asociados a parcelas y fechas, que aportan trazabilidad y contexto a la información.

Estas fuentes se evaluaron no solo por su valor analítico, sino también por su facilidad de integración en la plataforma y su impacto en la experiencia de usuario.

5.3 Preparación de la información para su uso en la web

Una parte clave del trabajo fue preparar la información para que pudiera usarse de forma fluida dentro de una aplicación web, y no únicamente para análisis internos. Esto implicó decisiones conjuntas sobre:

- qué nivel de detalle era necesario mostrar,
- con qué frecuencia debían actualizarse los datos,
- cómo asociarlos a parcelas, fechas y cultivos,
- y cómo garantizar una trazabilidad mínima de la información.

Los datos preparados se estructuran y se exponen como servicios accesibles mediante APIs, lo que permite que el backend los gestione de forma homogénea y que el frontend los consuma sin depender de los detalles internos del procesamiento.

5.4 Desarrollo de servicios de inteligencia

La inteligencia del sistema se ha implementado en forma de servicios independientes, accesibles mediante APIs. Este enfoque permite que la plataforma consuma análisis y resultados sin estar acoplada a cómo se generan internamente.

Desde el punto de vista del equipo, esto ha permitido un reparto claro de responsabilidades: el equipo de Data Science desarrolla los servicios analíticos y de interpretación, y el equipo Full Stack los integra en la plataforma, construyendo flujos de uso coherentes y comprensibles.

Este diseño facilita además la evolución futura del sistema, permitiendo mejorar o sustituir servicios sin afectar al conjunto de la aplicación.

5.5 Validación dentro del producto

La validación del trabajo con datos no se ha centrado únicamente en comprobar que los cálculos o análisis fueran correctos, sino en verificar que la información resultante tenía sentido dentro del uso real de la plataforma.

Se evaluó, por ejemplo, si:

- las alertas llegaban en el momento adecuado,
- los resultados eran comprensibles para cada perfil,
- y la información facilitaba la comunicación entre los distintos actores.

Este proceso de validación conjunta ha permitido ajustar tanto los servicios de datos como su presentación en la interfaz, reforzando el enfoque de producto usable desde el primer momento.

Diagrama 5 — Trabajo integrado con datos dentro de la plataforma



6. Inteligencia aplicada y resultados en el MVP

6.1 La inteligencia como parte del producto, no como fin en sí mismo

La inteligencia desarrollada en este proyecto no se concibe como un conjunto de modelos aislados, sino como una capa transversal que aporta valor a la plataforma. El objetivo no es mostrar tecnología, sino ayudar a tomar mejores decisiones en un entorno agrícola cada vez más complejo y condicionado por el cambio climático.

Desde el inicio, se ha trabajado con una idea clara: si un análisis o una predicción no puede entenderse, explicarse y utilizarse dentro de la plataforma, no tiene sentido incluirlo en el MVP.

Por eso, toda la inteligencia desarrollada se expone como servicios integrados, que el sistema utiliza en el momento adecuado y presenta de forma comprensible según el perfil del usuario.

6.2 Qué tipo de decisiones apoya la plataforma

La plataforma aporta apoyo a la decisión en varios niveles clave del día a día agrícola:

En primer lugar, permite anticipar situaciones de riesgo climático, como heladas u olas de calor, facilitando que productores y técnicos actúen antes de que el impacto sea irreversible.

En segundo lugar, ayuda a identificar y documentar problemas en el cultivo, como la aparición de plagas o el deterioro del estado del campo, apoyándose en evidencias visuales reales y no solo en descripciones subjetivas.

Además, ofrece una visión compartida del estado productivo, lo que resulta especialmente útil para managers a la hora de planificar logística, ajustar volúmenes o gestionar incidencias en la cadena de suministro.

6.3 Uso de datos climáticos y visuales en el MVP

La inteligencia del sistema se apoya principalmente en dos tipos de información:

Por un lado, los datos climáticos, fundamentales en un contexto de cambio climático. Estos datos permiten contextualizar lo que ocurre en el campo, entender por qué se produce un problema y anticipar escenarios de riesgo.

Por otro lado, las imágenes capturados en campo, que aportan una visión directa y objetiva del estado real del cultivo. Estas evidencias visuales permiten apoyar diagnósticos técnicos, mejorar la comunicación entre actores y dejar constancia de incidencias de forma trazable.

Ambos tipos de información se combinan dentro de la plataforma para ofrecer una visión más completa y fiable de la situación del campo.

6.4 Resultados obtenidos en el MVP

En el alcance actual del MVP, la plataforma ha demostrado ser capaz de:

- Generar alertas tempranas ante situaciones de riesgo relevantes.
- Integrar información climática y visual en un único entorno.
- Facilitar la comunicación y coordinación entre productores, asesores y managers.
- Aportar evidencias objetivas que respaldan decisiones técnicas y comerciales.

Más allá de métricas técnicas, el principal resultado del MVP es haber validado que la inteligencia integrada en la plataforma aporta valor real cuando se presenta de forma clara y contextualizada.

6.5 Evolución prevista de la inteligencia del sistema

El diseño del sistema permite que esta capa de inteligencia evolucione de forma progresiva. Funcionalidades como la estimación del estado de maduración, el análisis de suelo mediante imágenes 360° o recomendaciones más avanzadas se han definido como siguientes pasos naturales, una vez validado el uso del producto y con mayor disponibilidad de datos y tiempo.

Diagrama 6 — Inteligencia integrada en la plataforma

Diagrama 6 — Inteligencia integrada en la plataforma



7. Validación, límites del MVP y líneas de evolución

7.1 Cómo se ha validado el MVP

La validación del proyecto se ha realizado desde una perspectiva práctica y realista, coherente con el carácter de producto piloto (MVP). El objetivo no era demostrar un sistema completamente cerrado, sino comprobar que la solución propuesta funciona, aporta valor y tiene sentido en un entorno real.

Desde el punto de vista técnico, se ha validado que la plataforma es capaz de integrar datos climáticos, imágenes de campo y servicios de inteligencia dentro de una arquitectura web estable, modular y basada en APIs. Los distintos componentes se comunican correctamente y permiten que la información fluya desde las fuentes de datos hasta los usuarios finales.

Desde el punto de vista funcional y de negocio, la validación se ha apoyado en escenarios reales planteados por el cliente, como la gestión de una helada que afecta a la producción. En este tipo de situaciones, la plataforma demuestra su utilidad como punto de coordinación entre productores, asesores y managers, facilitando la toma de decisiones basada en datos y evidencias compartidas.

El principal resultado de esta validación es confirmar que el enfoque elegido resuelve un problema real del sector y que la plataforma puede utilizarse como base para un producto comercial.

7.2 Límites actuales del MVP

El proyecto se ha desarrollado en un marco temporal limitado, propio de un MVP cuyo objetivo principal es validar la idea y priorizar funcionalidades clave. Por este motivo, se han tomado decisiones conscientes para acotar el alcance y centrarse en aquello que aporta mayor valor inmediato.

Entre los límites actuales del MVP se encuentran:

- Cobertura limitada de cultivos, plagas y escenarios productivos.
- Funcionalidades de análisis y predicción centradas en los casos más relevantes.
- Dependencia de la disponibilidad y calidad de datos externos, especialmente climáticos y visuales.

- Ausencia de sistemas avanzados de recomendación automática, que requieren mayor volumen de datos y validación en campo.

Estos límites no representan carencias del sistema, sino decisiones de diseño propias de un producto en fase inicial, orientadas a asegurar robustez, claridad y facilidad de uso.

7.3 Líneas de evolución y mejora del producto

Una vez validado el MVP, la plataforma presenta un amplio margen de evolución hacia funcionalidades más avanzadas, siempre manteniendo el enfoque de herramienta de apoyo a la decisión.

Entre las principales líneas de evolución se incluyen:

La gestión inteligente del riego, integrando información sobre estado del suelo, condiciones climáticas y estado del cultivo, con el objetivo de optimizar el uso del agua y mejorar la eficiencia productiva.

El desarrollo de recomendaciones agronómicas avanzadas, orientadas a apoyar decisiones sobre uso de fitosanitarios, fertilizantes y planes de nutrición, siempre con supervisión humana y adaptadas al contexto del cultivo.

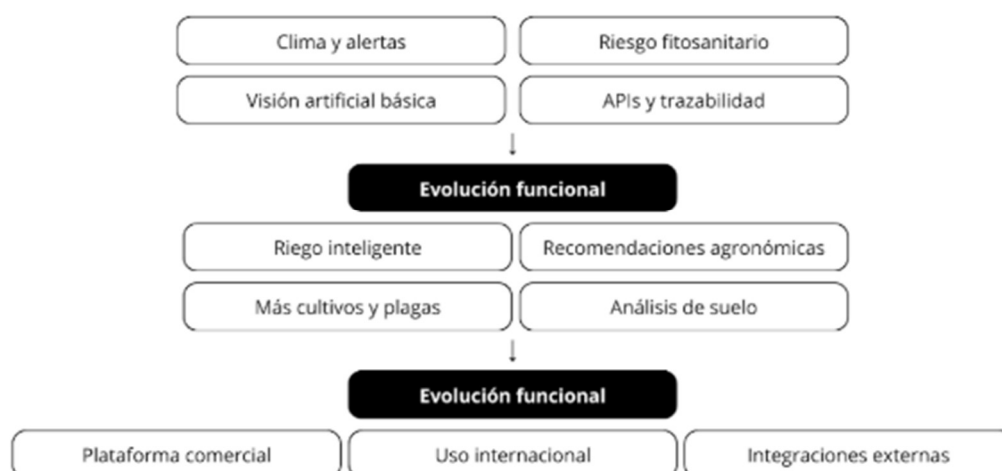
La ampliación del análisis visual y del estado del cultivo, incorporando estimación de maduración, análisis mediante imágenes 360° y mayor detalle sobre suelo y productividad.

El escalado funcional y geográfico del sistema, adaptando la plataforma a nuevos cultivos, otros países y distintos contextos climáticos, así como su integración con sistemas externos del sector.

Estas líneas de evolución refuerzan el posicionamiento del proyecto como una base sólida para un producto agrícola digital, capaz de crecer de forma progresiva según las necesidades del sector y la disponibilidad de recursos.

Diagrama 7 — Evolución del MVP hacia producto completo

Diagrama 7 — Evolución del MVP hacia producto completo



8 Conclusiones

El proyecto desarrollado demuestra que es posible transformar una necesidad real del sector frutícola en una solución digital coherente, viable y con visión de producto, incluso en un contexto de tiempo limitado y con un alcance propio de un MVP.

Partiendo de un problema concreto -la gestión de campos agrícolas en un entorno marcado por el cambio climático, la incertidumbre y la coordinación entre múltiples actores-, se ha construido una plataforma web de inteligencia agrícola que integra datos, análisis y evidencias visuales en un único entorno accesible y comprensible.

El valor de la solución no reside únicamente en la tecnología utilizada, sino en cómo se ha aplicado. El trabajo conjunto de los equipos de Full Stack y Data Science ha permitido convertir modelos, datos y servicios analíticos en funcionalidades reales dentro de una plataforma usable, alineada con las necesidades de productores, asesores y managers.

Desde el punto de vista técnico, el proyecto valida una arquitectura modular basada en APIs, preparada para evolucionar, escalar e integrarse con otros sistemas. Desde el punto de vista funcional y de negocio, la plataforma aporta información objetiva, trazable y compartida, facilitando la toma de decisiones y la coordinación en escenarios normales y de crisis.

Este MVP no se presenta como un producto cerrado, sino como una base sólida sobre la que construir. Las líneas de evolución definidas -riego inteligente, recomendaciones agronómicas, análisis avanzado del cultivo y escalado internacional- refuerzan su potencial como solución comercial adaptable a distintos contextos agrícolas.

En conjunto, el proyecto refleja la capacidad del equipo para analizar un problema real, diseñar una solución tecnológica con criterio y ejecutarla de forma coordinada, demostrando madurez profesional, visión de producto y capacidad de trabajo multidisciplinar.

Anexo 1

DOCUMENTACIÓN TÉCNICA – APIS CULTITECH

Sistema de Gestión y Control de Campos de Producción Agrícola

Este documento técnico documenta **APIs especializadas** en gestión integral de campos agrícolas:

1. [CLIMA EN TIEMPO REAL / DATOS DE CLIMA SEMANA PREVIA – Datos meteorológicos en tiempo real](#)
2. [AGROMETEO HISTÓRICOS – Análisis de tendencias históricas](#)
3. [ALERTA DE PLAGAS – Predicción de riesgos fitosanitarios](#)
4. [DATOS CLIMÁTICOS – Resumen diario de indicadores](#)
5. [ALERTA ML – Predicción con Machine Learning](#)
6. [DETECCIÓN DE PLAGAS – Visión artificial para identificación](#)
7. [CLASIFICACIÓN DE PLANTAS – Identificación de especies](#)
8. [ANÁLISIS DE SUELO – Propiedades y productividad](#)
9. [ASISTENTE IA – Asesoramiento inteligente](#)
10. [AURAVANT – Creación y consulta de parcelas. \(DEPRECADA\)](#)
11. [OPEN-METEO - Proporciona resumen diario completo de indicadores meteorológicos claves para la gestión efectiva de cultivos\(histórico 5 años\)](#)

Todas las APIs están documentadas bajo **formato unificado**, con **nomenclatura consistente**, **estilos visuales uniformes** y **criterio** para facilitar integración y comprensión por equipos técnicos y no técnicos.

API 1: CLIMA EN TIEMPO REAL / DATOS DE CLIMA SEMANA PREVIA

Descripción General

API diseñado con siete endpoints para consultar datos meteorológicos en **tiempo real de múltiples variables climáticas**. Un octavo endpoint extrae por defecto datos de medias diarias de la semana anterior al día actual, configurable por parámetro hasta 90 días.

Proporciona información integral para monitorización agrícola de parcelas específicas, integrando datos de la plataforma Open-Meteo con almacenamiento automático de ubicaciones.

Información Técnica Base

Característica	Detalle
Tecnología	Flask + Open-Meteo API
Autenticación	No requerida
Formato de datos	JSON
Tipo de respuesta	JSON
Puerto	18.207.240.23

Endpoint Principal de Monitorización

URL: <http://34.201.98.55/>

Método HTTP: GET

Descripción: Verifica el estado y disponibilidad de la API

Endpoints de Consulta de datos en tiempo real

Endpoint	Variable	Unidad	Descripción
/temperatura	Temperatura del aire	°C	Temperatura ambiente a 2 metros de altura
/humedad_relativa	Humedad relativa	%	Humedad relativa ambiente a 2 metros de altura
/humedad_suelo	Humedad del suelo	m³/m³	Humedad a 0.1 cm de profundidad

/precipitacion	Precipitación (lluvia/nieve)	mm	Volúmen total de agua que cae en la superficie
/viento_velocidad	Velocidad del viento	km/h	Velocidad del viento a 10 metros de altura
/viento_direccion	Dirección del viento	° (0-360)	Dirección del viento a 10 metros de altura
/evapotranspiracion	Evapotranspiración	mm	Cantidad de agua que el suelo y las plantas pierden hacia la atmósfera

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Ejemplo
parcela_id	Integer	Sí	Identificador único de la parcela	784112
lat	Float	Sí	Latitud de la parcela	-33.1951
lon	Float	Sí	Longitud de la parcela	-70.7292

Formato de Consulta (Entrada)

```
{
  "parcela_id": 784112,
  "lat": -33.1951,
  "lon": -70.7292
}
```

Formato de Respuesta (Salida)

```
{
  "parcela": {
    "lat": -33.1951,
    "lon": -70.7292,
    "parcela_id": 784112
  },
  "status": "Guardado y Monitorizado",
}
```

```
"valor": 15.1,
"variable": "temperatura"
}
```

Campos de Respuesta

Campo	Tipo	Descripción
parcela.lat	Float	Latitud de la parcela consultada
parcela.lon	Float	Longitud de la parcela consultada
parcela.parcela_id	Integer	Identificador único de la parcela
status	String	Estado de la operación ("Guardado y Monitorizado")
valor	Float	Valor numérico de la variable meteorológica
variable	String	Nombre de la variable consultada

Códigos de Estado HTTP

Código	Significado	Descripción
200	OK	Envía POST a los endpoints climáticos con JSON de la parcela
400	Error de solicitud	Parámetros inválidos o incompletos
500	Error de servidor	Problema interno del servidor

Casos de Uso

- Monitoreo en tiempo real de 7 variables climáticas.
- Seguimiento de humedad de suelo para optimizar riego
- Análisis de precipitación acumulada diaria
- Predicción de condiciones adversas mediante velocidad y dirección del viento
- Cálculo de necesidades de evapotranspiración para gestión hídrica

Notas Técnicas

- Los datos se actualizan en tiempo real desde Open-Meteo cada 60 minutos
- La parcela se almacena automáticamente en el sistema de monitorización tras la primera consulta
- Las coordenadas deben estar en rangos válidos: latitud (-90 a 90), longitud (-180 a 180)
- Los datos meteorológicos tienen precisión de una posición decimal
- Se recomienda consultar este endpoint diariamente para mantener históricos actualizados

Endpoint de Consulta de valores medios diarios la semana previa al día de consulta

Endpoint	Variables (dato diario)	Descripción
/consulta_datos	"temperature_2m" "relative_humidity_2m" "soil_moisture_0_1cm" "precipitation" "windspeed_10m" "winddirection_10m" "evapotranspiration"	Historial de datos climáticos diarios a partir de la fecha actual, los 7 días anteriores por defecto, por parámetro hasta 90 días antes como máximo.

API 2: AGROMETEO HISTÓRICOS – EXTRACCIÓN DE DATOS HISTÓRICOS (DEPRECADO)

Descripción General

API especializado en extracción de **datos meteorológicos históricos en rangos de fechas específicas**. Permite crear visualizaciones gráficas, análisis de tendencias y estudios comparativos de variables climáticas para optimización agrícola.

Información Técnica Base

Característica	Detalle
Tecnología	Flask + Open-Meteo API
Autenticación	No requerida
Formato de datos	JSON
Tipo de respuesta	JSON
Puerto	34.201.98.55

Endpoint Principal de Monitorización

URL: <http://34.201.98.55/>

Método HTTP: GET

Descripción: Verifica el estado y disponibilidad de la API

Endpoint de Consulta

URL: http://34.201.98.55/cargar_historico

Método HTTP: POST

Descripción: Extrae datos históricos de variables meteorológicas en rango de fechas

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Ejemplo
parcela_id	Integer	Sí	Identificador único de la parcela	784112
lat	Float	Sí	Latitud de la parcela	-33.1951
lon	Float	Sí	Longitud de la parcela	-70.7292
inicio	String (YYYY-MM-DD)	Sí	Fecha de inicio del período	"2024-01-01"
fin	String (YYYY-MM-DD)	Sí	Fecha de fin del período	"2024-01-15"
variables	Array	Sí	Variables a consultar	["temperatura", "precipitacion"]

Formato de Consulta (Entrada)

```
{  
  "parcela_id": 784112,  
  "lat": -33.1951,  
  "lon": -70.7292,  
  "inicio": "2024-01-01",  
  "fin": "2024-01-15",  
  "variables": [  
    "temperatura",  
    "precipitacion",  
    "humedad_suelo",  
    "evapotranspiracion"  
  ],  
  "humedad_relativa",  
  "viento_velocidad",  
  "viento_direccion"  
}
```

Formato de Respuesta (Salida)

```
{  
  "historico": [  
    {
```

```

"fecha": "2024-01-01",
"temperatura": 21.1,
"precipitacion": 0.0,
"humedad_suelo": 0.245,
"evapotranspiracion": 7.15
},
{
"fecha": "2024-01-02",
"temperatura": 22.0,
"precipitacion": 0.0,
"humedad_suelo": 0.247,
"evapotranspiracion": 7.36
}
],
"parcela_id": 784112,
"periodo": {
"inicio": "2024-01-01",
"fin": "2024-01-15"
},
"status": "Éxito",
"total_dias": 15
}

```

Campos de Respuesta

Campo	Tipo	Descripción
historico	Array	Lista de registros diarios ordenados cronológicamente
historico[].fecha	String	Fecha del registro en formato YYYY-MM-DD
historico[].temperatura	Float	Temperatura promedio del día (°C)
historico[].precipitacion	Float	Precipitación acumulada del día (mm)
historico[].humedad_suelo	Float	Humedad volumétrica del suelo (m³/m³)
historico[].evapotranspiracion	Float	Evapotranspiración diaria (mm)
historico[].humedad_relativa	Float	% de humedad en el aire
historico[].viento_velocidad	Float	Velocidad del viento en km/h
historico[].viento_direccion	Float	Origen del viento
parcela_id	Integer	Identificador de la parcela consultada

periodo.inicio	String	Fecha de inicio del período solicitado
periodo.fin	String	Fecha de fin del período solicitado
status	String	Estado de la operación ("Éxito" o "Error")
total_dias	Integer	Número total de días en el período

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Datos históricos consultados correctamente
400	Error de solicitud	Parámetros inválidos o formato incorrecto
404	No encontrado	Sin datos disponibles para el período solicitado
500	Error de servidor	Problema interno del servidor

Casos de Uso

- Análisis de tendencias climáticas a largo plazo
- Generación de gráficos de evolución de humedad, temperatura y demás variables
- Comparativa de ciclos agrícolas entre años
- Cálculo de acumulados de precipitación y evapotranspiración
- Auditoría de condiciones climáticas en período de cultivo

Notas Técnicas

- Los datos históricos están disponibles desde 1950 (según Open-Meteo)
- El período máximo recomendado es 365 días para optimizar rendimiento
- Los datos se devuelven ordenados cronológicamente (ascendente: de más antiguo a más reciente)

API 3: SISTEMA DE ALERTA DE PLAGAS – MONITOREO DE RIESGO FITOSANITARIO

Descripción General

Microservicio especializado en predicción de riesgo de plagas mediante análisis de datos meteorológicos en tiempo real. Integra variables de temperatura, humedad y precipitación con modelos de Grados Día Desarrollo (GDD) para optimizar decisiones de tratamientos fitosanitarios en cultivos frutales.

Fórmula cálculo GDD

$$GDD = \max(0, \frac{T_{max} + T_{min}}{2} - T_{base})$$

Información Técnica Base

Característica	Detalle
Tecnología	Flask + Open-Meteo API + Modelos Predictivos
Autenticación	No requerida
Formato de datos	JSON
Tipo de respuesta	JSON
Puerto	44.207.1.251

Endpoint Principal de Monitorización

URL: <https://aanearana-deteccion-plagas.hf.space/>

Método HTTP: GET

Descripción: Verifica el estado y disponibilidad de la API

Endpoint de Consulta

URL: <https://aanearana-deteccion-plagas.hf.space/plagas>

Método HTTP: POST

Descripción: Obtiene análisis de riesgo de plagas para un cultivo en ubicación específica

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Ejemplo
latitud	Float	Sí	Latitud de la parcela	-60.0
longitud	Float	Sí	Longitud de la parcela	-50.0
fruta	String	Sí	Tipo de cultivo (ver cultivos soportados)	"manzana"

Formato de Consulta (Entrada)

<https://aanearana-deteccion-plagas.hf.space/plagas?latitud=-60&longitud=-50&fruta=manzana>

Formato de Respuesta (Salida)

```
{
  "alertas": [
    {
      "pest": "pulgón",
      "risk_level": "bajo",
      "risk_percent": 30.0,
      "gdd_7_dias": 0,
      "status": "Monitoreo preventivo recomendado."
    },
    {
      "pest": "moteado",
      "risk_level": "bajo",
      "risk_percent": 30.0,
      "gdd_7_dias": 0,
      "status": "Monitoreo preventivo recomendado."
    }
  ],
  "config": {
    "fruta": "manzana",
    "lat": -60.0,
    "lon": -50.0
  },
  "meteo_data": {
    "max_temp_semana": 3.0,
    "min_temp_semana": 0.4,
    "humedad_promedio_max": 96.0,
    "lluvia_detectada": "Sí"
  }
}
```

```

}
}

```

Campos de Respuesta

Campo	Tipo	Descripción
alertas	Array	Lista de alertas por plagas detectadas
alertas[].pest	String	Nombre de la plaga
alertas[].risk_level	String	Nivel de riesgo (bajo, medio, alto, crítico)
alertas[].risk_percent	Float	Porcentaje de riesgo de infestación (0-100)
alertas[].gdd_7_dias	Float	Grados Día Desarrollo acumulados en 7 días
alertas[].status	String	Recomendación o descripción del estado
config.fruta	String	Cultivo analizado
config.lat	Float	Latitud de la consulta
config.lon	Float	Longitud de la consulta
meteo_data.max_temp_semana	Float	Temperatura máxima de la semana (°C)
meteo_data.min_temp_semana	Float	Temperatura mínima de la semana (°C)
meteo_data.humedad_promedio_max	Float	Humedad relativa máxima promedio (%)
meteo_data.lluvia_detectada	String	Indicador de precipitación ("Sí" / "No")

Cultivos Soportados

Cultivo	Plagas Monitoreadas
Uva	Mosca de la fruta, Botrytis, Oídio
Cítricos (Naranja, Limón)	Pulgón, Cochinilla, Minador
Manzana	Moteado, Carpocapsa, Pulgón

Kiwi	Plagas específicas del kiwi
------	-----------------------------

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Análisis de plagas completado
400	Error de solicitud	Parámetros inválidos o incompletos
404	No soportado	Cultivo no soportado por la API
500	Error de servidor	Problema interno del servidor

Casos de Uso

- Predicción de riesgo de plagas para decisiones de tratamiento
- Optimización de aplicaciones de agroquímicos
- Alertas tempranas de infestación potencial
- Monitoreo continuo durante épocas de riesgo

Notas Técnicas

- Los datos meteorológicos se obtienen en tiempo real de Open-Meteo
- El modelo utiliza Grados Día Desarrollo (GDD) como indicador biológico
- El análisis se realiza para los últimos 7 días de datos disponibles
- Se recomienda monitoreo continuo durante épocas de riesgo (primavera-verano)
- Las coordenadas deben estar dentro de rangos agrícolas válidos

API 4: CONSULTA DE DATOS CLIMÁTICOS PARA ANÁLISIS AGRÍCOLA

Descripción General

API que proporciona un resumen diario completo de indicadores meteorológicos clave para la gestión efectiva de cultivos. Devuelve series de datos ordenadas cronológicamente durante un período determinado (por defecto semanalmente, pero se puede indicar el periodo deseado), optimizado para análisis y visualización.

Información Técnica Base

Característica	Detalle
Tecnología	Flask + Open-Meteo API
Autenticación	No requerida
Formato de datos	JSON
Tipo de respuesta	JSON
Puerto	98.82.122.18

Endpoint Principal de Monitorización

URL: <http://34.201.98.55/>

Método HTTP: GET

Descripción: Verifica el estado y disponibilidad de la API

Endpoint de Consulta

URL: http://34.201.98.55/consultar_datos

Método HTTP: GET

Descripción: Retorna resumen diario de indicadores agrícolas para período especificado

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Ejemplo
lat	Float	Sí	Latitud de la ubicación	-37.9739
lon	Float	Sí	Longitud de la ubicación	-72.5897

days	Integer	No	Número de días a consultar (7 días por defecto)	15
------	---------	----	---	----

Formato de Consulta (Entrada)

http://34.201.98.55/consultar_datos?lat=-37.9739&lon=-72.5897&days=15

Formato de Respuesta (Salida)

```
{
  "data": [
    {
      "date": "2026-01-15",
      "temp_mean": 14.77,
      "humidity_mean": 60.29,
      "precip_prob": 6.58,
      "evapotranspiration": 5.54,
      "wind_speed": 15.85,
      "wind_direction": 190.93
    },
    {
      "date": "2026-01-16",
      "temp_mean": 16.19,
      "humidity_mean": 50.04,
      "precip_prob": 0.0,
      "evapotranspiration": 6.75,
      "wind_speed": 19.67,
      "wind_direction": 209.89
    }
  ],
  "location": {
    "lat": -37.9739,
    "lon": -72.5897
  },
  "status": "ok"
}
```

Campos de Respuesta

Campo	Tipo	Descripción
data	Array	Lista de registros diarios

data[].date	String	Fecha del registro en formato YYYY-MM-DD
data[].temp_mean	Float	Temperatura media del día (°C)
data[].humidity_mean	Float	Humedad relativa media (%)
data[].precip_prob	Float	Probabilidad de precipitación (%)
data[].evapotranspiration	Float	Evapotranspiración diaria (mm)
data[].wind_speed	Float	Velocidad media del viento (km/h)
data[].wind_direction	Float	Dirección del viento (0-360°)
location.lat	Float	Latitud consultada
location.lon	Float	Longitud consultada
status	String	Estado de la operación ("ok" o "error")

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Datos climáticos consultados correctamente
400	Error de solicitud	Parámetros inválidos o incompletos
422	Error de validación	Valores fuera de rango válido
500	Error de servidor	Problema interno del servidor

Casos de Uso

- Análisis de condiciones meteorológicas para toma de decisiones agrícolas
- Cálculo de necesidades de riego basado en evapotranspiración
- Generación de reportes climáticos diarios
- La evapotranspiración es clave para optimizar riego y drenaje

Notas Técnicas

- El parámetro days por defecto es 7 si no se especifica

- Los datos se retornan ordenados cronológicamente (ascendente)
- Las coordenadas deben estar en rangos válidos: latitud (-90 a 90), longitud (-180 a 180)

API 5: MICROSERVICIO DE ALERTA AGROMETEOROLÓGICA CON MACHINE LEARNING

Descripción General

API avanzada basada en Flask que utiliza Machine Learning (modelos Random Forest) para predecir riesgos climáticos específicos para cultivos chilenos. Entrena modelos automáticamente con datos históricos (2015-2025) bajo demanda, combinando predicciones meteorológicas con reglas críticas de cada cultivo.

Información Técnica Base

Característica	Detalle
Tecnología	Flask + Open-Meteo API + Random Forest
Autenticación	No requerida
Formato de datos	JSON
Tipo de respuesta	JSON
Puerto	18.184.142.242

Endpoint Principal de Monitorización

URL: <http://18.184.142.242/>

Método HTTP: GET

Descripción: Verifica el estado y disponibilidad de la API

Endpoint de Predicción

URL: <http://18.194.33.82:5000/monitor>

Método HTTP: POST

Descripción: Calcula nivel de riesgo climático para los próximos 7 días

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Ejemplo
-----------	------	-----------	-------------	---------

lat	Float	Sí	Latitud del terreno (Chile: valores negativos)	-34.58
lon	Float	Sí	Longitud del terreno	-70.94
id	String	Sí	Identificador único para guardar el modelo entrenado	"Fundo_El_Cerezo_01"
cultivo	String	Sí	Llave del cultivo soportado	"cerezo"

Formato de Consulta (Entrada)

```
{
  "lat": -34.58,
  "lon": -70.94,
  "id": "Fundo_El_Cerezo_01",
  "cultivo": "cerezo"
}
```

Formato de Respuesta (Salida)

```
{
  "cultivo": "Cerezo",
  "id": "Fundo_El_Cerezo_01",
  "reporte_7_dias": [
    {
      "date": "Thu, 22 Jan 2026 00:00:00 GMT",
      "estado": "❄️ HELADA CRÍTICA",
      "t_max": -12.7,
      "t_min": -14.9,
      "hr_min": 82,
      "nieve_sum": 0.0,
      "viento_max": 6.2
    },
    {
      "date": "Fri, 23 Jan 2026 00:00:00 GMT",
      "estado": "❄️ HELADA CRÍTICA",
      "t_max": -11.5,
      "t_min": -14.9,
      "hr_min": 77,
      "nieve_sum": 0.0,
      "viento_max": 16.3
    }
  ]
}
```

```

]
}

```

Campos de Respuesta

Campo	Tipo	Descripción
cultivo	String	Nombre del cultivo analizado
id	String	Identificador único de la ubicación
reporte_7_dias	Array	Lista de predicciones diarias (próximos 7 días)
reporte_7_dias[].date	String	Fecha y hora del pronóstico (formato GMT)
reporte_7_dias[].estado	String	Estado del clima con emoji indicador
reporte_7_dias[].t_max	Float	Temperatura máxima predicha (°C)
reporte_7_dias[].t_min	Float	Temperatura mínima predicha (°C)
reporte_7_dias[].hr_min	Integer	Humedad relativa mínima (%)
reporte_7_dias[].nieve_sum	Float	Acumulación de nieve predicha (mm)
reporte_7_dias[].viento_max	Float	Velocidad máxima del viento (km/h)

Cultivos Soportados

Cultivo	Riesgos Monitoreados
Cerezo	Helada crítica, Golpe de calor, Helada negra, Viento fuerte
Palto	Helada crítica, Golpe de calor, Helada negra, Viento fuerte
Uva	Helada crítica, Golpe de calor, Helada negra, Viento fuerte
Nogal	Helada crítica, Golpe de calor, Helada negra, Viento fuerte
Otros	Helada crítica, Golpe de calor, Helada negra, Viento fuerte

Estados Posibles

Estado	Descripción	Condición
HELADA CRÍTICA	Temperatura bajo umbral de muerte del cultivo	$T < \text{umbral crítico}$
GOLPE DE CALOR	Temperaturas superiores a límite del cultivo	$T > 32-38^{\circ}\text{C}$
HELADA NEGRA	Frío extremo con baja humedad (sin escarcha)	$T < 0^{\circ}\text{C}$ y $\text{HR} < 70\%$
VIENTO FUERTE	Riesgo de caída de fruta o daño estructural	Viento $> 50 \text{ km/h}$
CONDICIONES ÓPTIMAS	Sin riesgos detectados	Todos parámetros normales

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Predicción completada correctamente
400	Error de solicitud	Parámetros inválidos o incompletos
404	No soportado	Cultivo no soportado
503	Servicio no disponible	Modelo aún se está cargando (esperar segundos)
500	Error de servidor	Problema interno del servidor

Casos de Uso

- Predicción de riesgos climáticos para cultivos frutales
- Alertas tempranas de riesgos agrometeorológicos
- Decisiones de protección de cosecha
- Planificación de operaciones agrícolas

Notas Técnicas

- Datos históricos descargados automáticamente: 10 años (2015-2025) de Open-Meteo
- Persistencia: modelos guardados en archivos .joblib para consultas futuras instantáneas
- Predicción a 7 días con actualización diaria

API 6: SISTEMA DE DETECCIÓN FITOSANITARIA CON VISIÓN ARTIFICIAL

Descripción General

API basado en Flask que utiliza visión artificial (YOLOv8) para identificar y clasificar plagas en cultivos a partir de imágenes. Permite auditorías de salud de cultivos en tiempo real, integrándose con herramientas de gestión agrícola para la rápida toma de decisiones

Información Técnica Base

Característica	Detalle
Tecnología	Flask + YOLOv8 + OpenCV
Autenticación	No requerida
Formato de datos	Multipart Form-Data (imagen) / JSON (respuesta)
Tipo de respuesta	JSON
URL	https://aanearana-modelo-deteccion-plagas.hf.space

Endpoint Principal de Monitorización

URL: <https://aanearana-modelo-deteccion-plagas.hf.space/>

Método HTTP: GET

Descripción: Verifica estado general de la API

Endpoints de Consulta

Endpoint	Método	Descripción
/detect	POST	Analiza imagen y detecta plagas
/status	GET	Verifica si modelo YOLOv8 está cargado

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Aceptados
file	Binary	Sí	Archivo de imagen binario de la planta/insecto	.jpg, .jpeg, .png, .bmp, .tiff, .webp

Formato de Consulta (Entrada)

Método: POST /detect

Content-Type: multipart/form-data

Body:

- file: [imagen binaria]

Formato de Respuesta (Salida)

```
{
  "detecciones": [
    {
      "plaga": "ácaros_rojos",
      "confianza": 0.95
    }
  ],
  "total": 1,
  "imagen_resultado": "/imagen/resultado_12345.jpg"
}
```

Campos de Respuesta

Campo	Tipo	Descripción
detecciones	Array	Lista de plagas detectadas en la imagen
detecciones[].plaga	String	Nombre de la clase detectada (definida en modelo)
detecciones[].confianza	Float	Confianza de la detección (0.0 - 1.0)
total	Integer	Número total de plagas detectadas

imagen_resultado	String	URL/ruta de la imagen anotada con detecciones
------------------	--------	---

Clases de plagas

Listado de plagas detectadas		
Indice	Código	Descripción
1	ID 0	Enrollador de la hoja del arroz
2	ID 1	Oruga de la hoja del arroz
3	ID 2	Mosca del tallo del arroz
4	ID 3	Barrenador asiático del arroz
5	ID 4	Barrenador amarillo del arroz
6	ID 5	Mosquito de las agallas del arroz
7	ID 6	Mosca del tallo del arroz (Stemfly)
8	ID 7	Saltamontes marrón (Brown plant hopper)
9	ID 8	Saltamontes de espalda blanca
10	ID 9	Pequeño saltamontes marrón
11	ID 10	Gorgojo del agua del arroz
12	ID 11	Saltahojas del arroz
13	ID 12	Trips del grano
14	ID 13	Plaga de la concha del arroz
15	ID 14	Larva de escarabajo (Grub)
16	ID 15	Grillo topo (Alacrán cebollero)
17	ID 16	Gusano de alambre
18	ID 17	Polilla de margen blanco
19	ID 18	Gusano cortador negro
20	ID 19	Gusano cortador grande
21	ID 20	Gusano cortador amarillo
22	ID 21	Araña roja
23	ID 22	Barrenador del maíz
24	ID 23	Gusano cogollero / Oruga militar
25	ID 24	Pulgones
26	ID 25	Escarabajo de las flores (Potosiabre vitarsis)
27	ID 26	Barrenador del melocotonero
28	ID 27	Pulgón inglés del grano
29	ID 28	Pulgón verde
30	ID 29	Pulgón de la avena y el cerezo
31	ID 30	Mosquito del trigo (Blossom midge)
32	ID 31	Ácaro de patas azules (Penthaleus major)
33	ID 32	Ácaro de patas largas
34	ID 33	Trips del trigo

35	ID 34	Mosca de sierra del trigo
36	ID 35	Minador del tallo del trigo (<i>Cerodonta denticornis</i>)
37	ID 36	Mosca de la remolacha
38	ID 37	Altica (<i>Escarabajo pulga</i>)
39	ID 38	Oruga de la col
40	ID 39	Gusano soldado de la remolacha
41	ID 40	Mosca de la mancha de la remolacha
42	ID 41	Polilla de los prados
43	ID 42	Gorgojo de la remolacha
44	ID 43	Escarabajo de mayo oriental
45	ID 44	Gorgojo de la alfalfa
46	ID 45	Oruga de las yemas del lino
47	ID 46	Chinche de la alfalfa
48	ID 47	Chinche de campo manchada
49	ID 48	Saltamontes / Langostas (<i>Locustoidea</i>)
50	ID 49	Escarabajo ampolla (<i>Lytta polita</i>)
51	ID 50	Escarabajo de las leguminosas
52	ID 51	Aceitero (<i>Escarabajo de la patata/ampolla</i>)
53	ID 52	Pulgón manchado de la alfalfa
54	ID 53	Trips de la alfalfa
55	ID 54	Trips (General)
56	ID 55	Avispilla de la semilla de alfalfa
57	ID 56	Mariposa de la col (<i>Pieris canidia</i>)
58	ID 57	Chinche verde de las plantas
59	ID 58	Polilla de la taza (<i>Limacodidae</i>)
60	ID 59	Filoxera de la vid
61	ID 60	Ácaro de la eriosis de la vid
62	ID 61	Ácaro de Lewis
63	ID 62	Escarabajo de diez puntos
64	ID 63	Ácaro blanco (<i>Araña blanca</i>)
65	ID 64	Cochinilla de Comstock
66	ID 65	Polilla avispa del chopo
67	ID 66	Esfinge de la vid (<i>Ampelophaga</i>)
68	ID 67	Mosca linterna con manchas
69	ID 68	Escarabajo de cuernos largos (<i>Barrenador de la madera</i>)
70	ID 69	Saltahoja verde
71	ID 70	Míridos (<i>Chinches de las plantas</i>)
72	ID 71	Mosca blanca de los invernaderos
73	ID 72	Saltahoja de la vid (<i>Erythroneura</i>)
74	ID 73	Mariposa cola de golondrina (<i>Papilio xuthus</i>)
75	ID 74	Ácaro rojo de los cítricos
76	ID 75	Ácaro del tostado de los cítricos
77	ID 76	Cochinilla acanalada
78	ID 77	Cochinilla de flecha de los cítricos

79	ID 78	Cochinilla de la cera roja
80	ID 79	Cochinilla circular de Florida
81	ID 80	Cochinilla negra del olivo/cítricos
82	ID 81	Cochinilla harinosa de la vid
83	ID 82	Mosca blanca espinosa
84	ID 83	Mosca de la fruta de los cítricos
85	ID 84	Mosca oriental de la fruta
86	ID 85	Mosca de la fruta japonesa
87	ID 86	Oruga de la col / Gusano del tabaco
88	ID 87	Mariposa del aceite (Adristyrannus)
89	ID 88	Minador de la hoja de los cítricos
90	ID 89	Pulgón negro de los cítricos
91	ID 90	Pulgón del café / Cítricos
92	ID 91	Pulgón verde de los cítricos
93	ID 92	Trips del chile / Cítricos
94	ID 93	Mosca de las agallas (Dasineura)
95	ID 94	Saltahojas de las plantas (Lawana)
96	ID 95	Saltahojas de margen verde
97	ID 96	Gorgojo de la hoja del mango
98	ID 97	Barrenador de los brotes del mango
99	ID 98	Saltahojas de pico plano del mango
100	ID 99	Barrenador de tronco largo
101	ID 100	Gorgojo del fruto del mango
102	ID 101	Cicadélidos (Saltahojas en general)

Respuesta de Estado (GET /status)

```
{
  "modelo": "best.pt (personalizado)",
  "modelo_cargado": true,
  "status": "listo"
}
```

Especificaciones Técnicas

Especificación	Detalle
Límite de carga	16 MB (soporta fotos de alta resolución)
Umbral de confianza	conf=0.25 (balance: detecta plagas pequeñas sin falsos positivos)
Prioridad de modelos	best.pt (personalizado) > yolov8n.pt (fallback)
Validación de integridad	Solo carga si archivo > 1 MB

Multithreading	Sí (no bloquea respuesta API durante carga)
----------------	---

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Detección completada correctamente
400	Error de solicitud	No se envió archivo o archivo vacío
405	Método no permitido	GET en /detect
503	Servicio no disponible	Modelo aun cargando, espere segundos
500	Error de servidor	Decodificación de imagen o falla del modelo

Casos de Uso

- Inspección rápida de salud de cultivos
- Identificación inmediata de plagas en campo
- Documentación de hallazgos fitosanitarios
- Apoyo en decisiones de tratamiento
- Auditoría de propiedades agrícolas

Notas Técnicas

- Primera consulta: ~5-10 segundos (carga del modelo)
- Fallback automático a yolov8n.pt si best.pt no disponible

API 7: DETECCIÓN Y CLASIFICACIÓN DE PLANTAS

Descripción General

API especializada en identificar y clasificar especies de plantas a partir de imágenes. Utiliza modelos de visión artificial para proporcionar identificación binomial (científica), nombres comunes alternativos y nivel de precisión de la clasificación.

Información Técnica Base

Característica	Detalle
Tecnología	Flask + Modelos de Visión Artificial
Autenticación	No requerida
Formato de datos	Multipart Form-Data (imagen) / JSON (respuesta)
Tipo de respuesta	JSON
Puerto	18.194.33.82

Endpoint Principal de Monitorización

URL: <http://18.194.33.82/>

Método HTTP: GET

Descripción: Verifica el estado y disponibilidad de la API

Endpoint de Identificación

URL: <http://18.194.33.82/identificar>

Método HTTP: POST

Descripción: Identifica la especie de planta de la imagen proporcionada

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Aceptados
image	Binario	Sí	Archivo de imagen binario de la planta	.jpg, .jpeg, .png

Formato de Consulta (Entrada)

Método: POST /identificar

Content-Type: multipart/form-data

Body:

- image: [imagen binaria]

Formato de Respuesta (Salida)

```
{
  "nombre_cientifico": "Vaccinium corymbosum",
  "nombre_comun": "Arándano americano",
  "otros_nombres": [
    "Uvilla azul",
    "Mora azul"
  ],
  "precision": "85.49%",
  "status": "success"
}
```

Campos de Respuesta

Campo	Tipo	Descripción
nombre_cientifico	String	Nombre científico de la planta (nomenclatura binomial)
nombre_comun	String	Nombre común más conocido de la planta
otros_nombres	Array	Otros nombres comunes alternativos
precision	String	Porcentaje de confianza de la clasificación
status	String	Estado de la operación ("success" o "error")

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Planta identificada correctamente
400	Error de solicitud	No se envió imagen o imagen vacía
405	Método no permitido	Método HTTP incorrecto
415	Tipo no soportado	Tipo de archivo no soportado
500	Error de servidor	Problema en identificación o modelo

Casos de Uso

- Identificación de especies cultivadas desconocidas
- Detección de plantas silvestres invasoras
- Catalogación de biodiversidad en propiedades
- Apoyo en decisiones de gestión de vegetación
- Educación y referencia agrícola

Notas Técnicas

- La precisión depende de calidad e iluminación de la imagen
- Se recomienda fotografías con buena claridad y enfoque
- Soporta plantas cultivadas y silvestres
- Modelo incluye flora de diferentes regiones geográficas
- Mejor rendimiento con imágenes de hojas/flores visibles

API 8: INFORMACIÓN Y ANÁLISIS DE SUELO

Descripción General

API que proporciona análisis geoespacial completo del suelo incluyendo propiedades químicas (pH, materia orgánica), físicas (textura), topográficas (elevación, pendiente) y productividad mediante índice de vegetación normalizado (NDVI) derivado de satélites MODIS. Los datos se extraen de Google Earth Engine.

Información Técnica Base

Característica	Detalle
Tecnología	Flask + Análisis Geoespacial + NDVI (MODIS)
Autenticación	No requerida
Formato de datos	JSON
Tipo de respuesta	JSON
Puerto	44.207.1.251

Endpoint Principal de Monitorización

URL: <http://44.207.1.251/health>

Método HTTP: GET

Descripción: Verifica el estado y disponibilidad de la API

Endpoint de Análisis

URL: <http://44.207.1.251/tierra>

Método HTTP: POST

Descripción: Retorna análisis completo de propiedades del suelo

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Ejemplo
parcela_id	Integer	Sí	Identificador único de la parcela	784112
lat	Float	Sí	Latitud de la ubicación	-33.8688

lon	Float	Sí	Longitud de la ubicación	-70.7000
-----	-------	----	--------------------------	----------

Formato de Consulta (Entrada)

```
{
  "parcela_id": 101,
  "lat": -33.4489,
  "lon": -70.6693
}
```

Formato de Respuesta (Salida)

```
{
  "coordenadas": {
    "lat": -33.8688,
    "lon": -70.7000
  },
  "suelo": {
    "ph_superficie": 6.5,
    "ph_profundidad": 6.8,
    "materiaorganica_gkg": 35,
    "textura": {
      "arena": 45,
      "limo": 35,
      "arcilla": 20
    }
  },
  "topografia": {
    "elevacion_msnm": 450,
    "pendiente_grados": 8.5
  },
  "clima": {
    "precipitacion_anual_mm": 380
  },
  "productividad_ndvi": {
    "2021": 0.62,
    "2022": 0.65,
    "2023": 0.67,
    "2024": 0.68,
    "2025": 0.70
  }
}
```

}

}

Campos de Respuesta

Campo	Tipo	Descripción
suelo.ph_superficie	Float	pH del suelo en superficie (0-14)
suelo.materia_organica_gkg	Float	Materia orgánica en g/kg
suelo.textura.arena	Float	Porcentaje de arena (%)
suelo.textura.limo	Float	Porcentaje de limo (%)
suelo.textura.arcilla	Float	Porcentaje de arcilla (%)
topografia.elevacion_msnm	Float	Elevación sobre nivel del mar (metros)
topografia.pendiente_grados	Float	Pendiente del terreno (grados)
clima_anual.precipitacion_acumulada_mm	Float	Precipitación anual promedio (mm)
ndvi_actual	Float	Índice de Vegetación por Diferencia Normalizada (0-1)

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Análisis de suelo completado
400	Error de solicitud	Parámetros inválidos o incompletos
422	Error de validación	Coordenadas fuera de rango válido
500	Error de servidor	Problema interno del servidor

Casos de Uso

- Caracterización de suelos para planificación de cultivos
- Análisis de fertilidad del terreno
- Evaluación de erosión y pendiente
- Seguimiento de productividad temporal (NDVI)
- Toma de decisiones sobre gestión de suelo

Notas Técnicas

- NDVI: índice derivado de satélites MODIS (rango 0-1, mayor = más productivo)
- pH: medida de acidez/alcalinidad (6.0-7.0 = óptimo para mayoría cultivos)
- Textura: clasificación basada en proporción de partículas (arena, limo, arcilla)
- Datos geoespaciales agregados de múltiples fuentes satelitales
- Precisión espacial aproximada: 250-500 metros

API 9: ASISTENTE AGRÍCOLA INTELIGENTE CON IA

Descripción General

API inteligente especializada en agricultura que combina conocimientos técnicos agronómicos y comerciales mediante modelos de lenguaje avanzados (LLM Groq). Proporciona asesoramiento personalizado basado en consultas en lenguaje natural, manteniendo historial conversacional por usuario.

Información Técnica Base

Característica	Detalle
Tecnología	Flask + LangChain + Groq LLM + PostgreSQL
Autenticación	No requerida
Formato de datos	JSON
Tipo de respuesta	JSON
Estado	☒ Activo

Endpoint Principal

URL: <https://vegetai-labels.onrender.com/chat>

Método HTTP: POST

Content-Type: application/json

Descripción: Procesa preguntas agrícolas y retorna asesoramiento inteligente

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Ejemplo
user_email	String	Sí	Email del usuario (identificador único para memoria conversacional)	" ejemplo@ejemplo.com "
message	String	Sí	Pregunta o consulta del usuario sobre temas agrícolas	"¿Qué cultivos hay en Chile en verano?"

Formato de Consulta (Entrada)

```
{  
  "user_email": "ejemplo@ejemplo.com",  
  "message": "¿Qué cultivos hay en Chile en verano?"  
}
```

Formato de Respuesta (Salida)

```
{  
  "response": "En Chile, el verano es una época ideal para cultivar una variedad de frutas y verduras.  
Las principales opciones son: Frutas (Uvas, Manzanas, Paltas, Fresas, Melones) y Verduras  
(Tomates, Pepinos, Calabacines, Cebollas, Lechugas). También se cultivan Maíz, Girasoles y  
Alfalfa. La disponibilidad varía según la región y condiciones climáticas. ¿Necesitas más  
información sobre cultivos específicos?",  
  "user_email": "ejemplo@ejemplo.com"  
}
```

Campos de Respuesta

Campo	Tipo	Descripción
response	String	Respuesta en formato texto del asistente IA
user_email	String	Email del usuario (para referencia)

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Respuesta generada correctamente
400	Error de solicitud	Faltan campos obligatorios en JSON
405	Método no permitido	Método HTTP incorrecto
500	Error de servidor	Problema interno del servidor o LLM

Características Técnicas

Característica	Detalle
Memoria conversacional	Mantiene historial por usuario (user_email)
Modelo LLM	Groq (respuestas rápidas y especializadas)
Base de datos	PostgreSQL para persistencia
Contexto agrícola	Entrenado con conocimientos especializados
Idioma	Español
Latencia típica	< 15 segundos por respuesta

Casos de Uso

- Consultas sobre cultivos apropiados para región
- Asesoramiento sobre plagas y control de enfermedades
- Recomendaciones de técnicas agrícolas
- Información sobre normativas y regulaciones
- Análisis de viabilidad de cultivos
- Apoyo en decisiones de comercialización
- Gestión de agua y sistemas de riego
- Análisis de suelo y fertilización
- Comercialización de productos
- Normativas agrícolas y fitosanitarias
- Sostenibilidad y agricultura responsable

Notas Técnicas

- Cada usuario identificado por email tiene su propio historial
- El modelo aprende el contexto de conversaciones anteriores
- Respuestas personalizadas según región y cultivos mencionados
- Base de datos PostgreSQL almacena conversaciones para continuidad
- LLM Groq proporciona respuestas especializadas en agricultura
- Sistema de contexto agrícola integrado para precisión

API 10: AURAVANT – GESTION DE CAMPOS CULTIVO (DEPRECADA)

Descripción General

API diseñada para crear las parcelas en base a las coordenadas que las delimitan (POLÍGONO), introducidas por el usuario, y la consulta de las mismas en AURAVANT con el ID de la parcela en AURAVANT que se genera al registrarlas (crearlas).

Información Técnica Base

Característica	Detalle
Tecnología	Flask + API AURAVANT
Autenticación	Token (asociado al usuario AURAVANT)
Formato de datos	JSON
Tipo de respuesta	JSON
Estado	5005

Endpoint Principal de Monitorización

URL: <https://api-auravant.onrender.com/>

Método HTTP: GET

Descripción: Verifica el estado y disponibilidad de la API

Respuesta: "API de aplicación campos agrícolas"

Endpoints de API

Endpoint	Función
/agregar_parcela	Crear parcela/campo
/consultar_parcela	Consultar parcela

Crear parcela/campo

Endpoint de creación de parcela

URL: https://api-auravant.onrender.com/agregar_parcela

Método HTTP: POST

Descripción: creación en AURAVANT de las parcelas introducidas por el usuario a partir de las coordenadas del polígono definido por el mismo. El usuario introducirá también el nombre de la parcela y el campo al que está asociada. Se pueden asociar varias parcelas en el mismo campo.

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Ejemplo
nombre_parcela	String	Sí	Nombre de la parcela que se registra	"TEST parcela"
coordenadas	String	Sí	Coordenadas de los puntos que delimitan la parcela (Polígono) en formato EPSG 4326 (WGS84): latitud, longitud (Y, X)	"POLYGON((-71.088430 - 34.727273, -71.087058 - 34.727273, -71.087058 - 34.727777, -71.088430 - 34.727777, -71.088430 - 34.727273))"
nombre_campo	String	Sí	Nombre del campo donde se ubica la parcela	"TEST campo 20260121005"

Formato de Consulta (Entrada)

```
{
  "data": [
    "TEST parcela 20260121005",
    "POLYGON((-71.088430 -34.727273, -71.087058 -34.727273, -71.087058 -34.727777, -71.088430 -34.727777, -71.088430 -34.727273))",
    "TEST campo 20260121005"
  ]
}
```

Formato de Respuesta (Salida)

```
{
  "id_campo": 227823,
  "id_lote": "784377",
  "info": "Nuevo id lote 784377",
  "res": "ok",
  "uuid_campo": "ed4a7eae-5b64-417b-b22a-ce620e0bd1cf",
  "uuid_lote": "7dc09229-0063-410e-8d90-c020aa5c1f8d"
}
```

Campos de Respuesta

Campo	Tipo	Descripción
id_campo	Integer	ID campo donde se agrupa la parcela creada
id_lote	String	ID en AURAVANT de la parcela creada
info	String	Mensaje informativo sobre el resultado de la operación
res	String	Estado de la operación ("ok" si la operación es correcta)
uuid_campo	String	UUID (Universally Unique Identifier) del campo creado.
uuid_lote	String	UUID (Universally Unique Identifier) de la parcela creada.

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Parcela creada correctamente en AURAVANT
400	Error de solicitud	Parámetros inválidos o incompletos
500	Error de servidor	Problema interno del servidor

Casos de Uso

- El usuario productor delimita en la aplicación las coordenadas del polígono de la parcela e introduce los nombres de la parcela (opcionalmente del campo, si se quiere asociar a otro existente).
- FRONT envía estos datos a la API /agregar_parcela (BACK).
- El BACK devuelve los datos de la parcela creada.

Consultar parcela

Endpoint de consulta de parcela

URL: https://api-auravant.onrender.com/consulta_parcela

Método HTTP: GET

Descripción: consulta en AURAVANT de la información de las parcelas introducidas por el usuario usando el ID de la parcela en AURAVANT.

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Ejemplo
-----------	------	-----------	-------------	---------

ID_parcela	String	Sí	ID de la parcela en AURAVANT que se quiere consultar	“784666”
------------	--------	----	--	----------

Formato de Consulta (Entrada)

```
{
  "data": [
    "784666"      #ID en AURAVANT de la parcela que se consulta
  ]
}
```

Formato de Respuesta (Salida)

```
{
  "country": "CL",
  "currency": "$",
  "farm": "test campo 20260122003",
  "id": 784666,
  "name": "TEST parcela 20260122003",
  "permissions": [
    3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 20, 22, 24, 27, 28,
    30, 31, 32, 33, 34, 35, 37, 38, 41, 42, 48, 50, 52, 53, 60, 61, 62, 63, 70, 72,
    94, 100, 101, 110, 111, 112, 113, 114, 120, 121, 131, 140, 141, 142, 143, 150,
    151, 210, 211, 212, 213, 220, 241, 250, 251, 252, 260, 261, 262, 320, 321, 322,
    323, 345, 800, 801, 802, 803, 804, 805, 806, 900, 901, 902, 903, 904, 905, 906,
    907, 909, 910, 911, 921
  ],
  "province": null,
  "role": 1,
  "shapes": {
    "current": {
      "area": 0.7026615,
      "bbox": "POLYGON((-71.08843 -34.727777,-71.08843 -34.727273,-71.087058 -34.727273,-71.087058 -34.727777,-71.08843 -34.727777))",
      "campaign_from": null,
      "campaign_to": null,
      "centroid": "POINT(-71.087744 -34.727525)",
      "end_date": "current",
      "polygon": "POLYGON((-71.08843 -34.727273,-71.087058 -34.727273,-71.087058 -34.727777,-71.08843 -34.727777,-71.08843 -34.727273))",
      "shape_id": 808399,
      "start_date": "2026-01-22T21:46:00Z"
    }
  }
}
```

Campos de Respuesta

Campo	Tipo	Descripción
id_parcela	Integer	ID parcela consultada. Coincidirá con el campo de la request.
name	String	Nombre de la parcela consultada en AURAVANT.
role	Integer	ID del role del usuario que hace la consulta en la parcela.
permissions	[Integer]	Permisos AURAVANT del usuario que hace la consulta en la parcela.
farm	String	Nombre del campo en la que está incluida la parcela
country	String	Código de país.
currency	String	Moneda del país de la parcela.
area	Integer	Tamaño de la parcela en Ha.
shape_id	Integer	ID del campo 'shape' de la parcela en AURAVANT
start_date	String	Fecha de creación de la parcela.
end_date	String	Fecha de borrado de la parcela. Sí está activo es "current".
polygon	String	Coordenadas de los puntos que delimitan la parcela (Polígono) en formato EPSG 4326 (WGS84): latitud, longitud (Y, X)
centroid	String	Coordenadas del centroide de la parcela (Polígono) en formato EPSG 4326 (WGS84): latitud, longitud (Y, X)
bbox	String	Coordenadas del rectangulo que contiene la parcela (Polígono) en formato EPSG 4326 (WGS84): latitud, longitud (Y, X)

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Consulta ejecutada correctamente
400	Error de solicitud	Parámetros inválidos o incompletos
500	Error de servidor	Problema interno del servidor

Casos de Uso

- El usuario productor introduce en la aplicación el ID de la parcela en AURAVANT.
- FRONT envía estos datos a la API /consultar_parcela (BACK).
- El BACK devuelve los datos de la parcela consultada.

Notas Técnicas

- **Autenticación:** el api de AURAVANT se usa mediante la creación de un Token Bearer que se pide en cada llamada a la API de AURAVANT ["https://livingcarbontech.auravant.com/api/auth"](https://livingcarbontech.auravant.com/api/auth) mediante una función específica que lo guarda en una variable global para su uso durante la operación.
- **Variables entorno (.env):** para obtener los token se usan unas credenciales en AURAVANT que se informan mediante unas variables de entorno que se deben configurar en el despliegue en el servidor:
 - AURAVANT_USUARIO: usuario registrado en AURAVANT
 - AURAVANT_PASSWORD: password del usuario registrado en AURAVANT
 - AURAVANT_ESPACIO: espacio en AURAVANT donde está registrado el usuario
 - AURAVANT_SUBDOMAIN: subdomain en AURAVANT donde está registrado el usuario
 - AURAVANT_CLIENT_ID: ID del cliente en el espacio AURAVANT donde está registrado el usuario
 - AURAVANT_SECRET: Clave SECRET en el espacio AURAVANT donde está registrado el usuario
- **Formato Request:** para la llamada a la API de AURAVANT, además del TOKEN, se pasan los siguientes parámetros para especificar el formato de la llamada:

- Authorization: "Bearer {TOKEN_API}"
- Content-Type: "application/x-www-form-urlencoded",
- x-auth-s: "AURAVANT_ESPACIO"

API 11: OPEN-METEO API para cargar histórico desde hace 5 años

Descripción General

API que proporciona un resumen diario completo de indicadores meteorológicos clave para la gestión efectiva de cultivos. Devuelve series de datos ordenadas cronológicamente durante un período determinado (por defecto semanalmente, pero se puede indicar el periodo deseado), optimizado para análisis y visualización.

Información Técnica Base

Característica	Detalle
Tecnología	Flask + Open-Meteo API
Autenticación	No requerida
Formato de datos	JSON
Tipo de respuesta	JSON

Endpoint Principal de Monitorización

URL: <https://cultitech-5years.onrender.com>

Método HTTP: GET

Descripción: Verifica el estado y disponibilidad de la API

Endpoint de Consulta

URL: https://cultitech-5years.onrender.com/5years_history?lat=-37.9739&lon=-72.5897

Método HTTP: GET

Descripción: Retorna resumen mensual de indicadores agrícolas para un periodo de 5 años

Parámetros de Entrada

Parámetro	Tipo	Requerido	Descripción	Ejemplo
lat	Float	Sí	Latitud de la ubicación	-37.9739
lon	Float	Sí	Longitud de la ubicación	-72.5897

Formato de Consulta (Entrada)

https://cultitech-5years.onrender.com/5years_history?lat=-37.9739&lon=-72.5897

Formato de Respuesta (Salida)

```
{
  "data": [
    {
      "date": "2021-01-01",
      "evapo": 14.797775268554688,
      "hum_rel": 80.09652709960938,
      "hum_suelo": 0.38495001196861267,
      "precip": 51.400001525878906,
      "temp": 14.531999588012695,
      "viento_dir": 183.28173828125,
      "viento_vel": 11.649556159973145
    }, {
      "date": "2021-02-01",
      "evapo": 134.52171325683594,
      "hum_rel": 67.31459045410156,
      "hum_suelo": 0.3048854172229767,
      "precip": 10.0, "temp": 16.490482330322266,
      "viento_dir": 193.58262634277344,
      "viento_vel": 14.413050651550293
    }
  ],
  "location": {"lat": -37.9739, "lon": -72.5897},
  "status": "ok"
}
```

Códigos de Estado HTTP

Código	Significado	Descripción
200	Éxito	Datos climáticos consultados correctamente
400	Error de solicitud	Parámetros inválidos o incompletos

422	Error de validación	Valores fuera de rango válido
500	Error de servidor	Problema interno del servidor

Casos de Uso

- Análisis de condiciones meteorológicas para toma de decisiones agrícolas
- Cálculo de necesidades de riego basado en evapotranspiración
- Generación de reportes climáticos diarios
- La evapotranspiración es clave para optimizar riego y drenaje

Notas Técnicas

- El parámetro days por defecto es 7 si no se especifica
- Los datos se retornan ordenados cronológicamente (ascendente)
- Las coordenadas deben estar en rangos válidos: latitud (-90 a 90), longitud (-180 a 180)

MODELO DATOS

Para almacenar la información generada en el BACKEND por las APIS se usa una BBDD Postgres con las siguientes tablas de información:

- **CLIMA_ACTUAL:** guarda el resultado de las consultas a las APIS de información meteorológica en tiempo real.
- **SUELO_PARCELA:** guarda las consultas a la API de composición de tierra.
- **CLIMA_HIST_PARCELA:** guarda las consultas a las APIS de históricos diarios y mensuales.
- **LOGS_PROCESOS:** guarda logs sobre las consultas realizadas a las APIs de clima y suelo.
- **AGRO_CHAT_HISTORY:** guarda las conversaciones del asistente agrícola con IA.

ANEXO 2: Referencias técnicas y recursos externos

ÍNDICE DEL ANEXO 2 – Referencias técnicas y recursos externos

2.1 Fuentes de APIs y Servicios Externos Integrados

Tabla 2.1 – APIs y servicios externos integrados en la plataforma

2.2 Tecnologías y Frameworks – Stack Técnico

2.2.1 Tecnologías Frontend

Tabla 2.2.1 – Tecnologías frontend del sistema

2.2.2 Tecnologías Backend

Tabla 2.2.2 – Tecnologías backend del sistema

2.2.3 Microservicios de Data Science

Tabla 2.2.3 – Tecnologías de Data Science e Inteligencia Artificial

2.2.4 Persistencia de Datos

Tabla 2.2.4 – Tecnologías de persistencia y gestión de datos

2.3 Visión Artificial y Detección de Objetos

Tabla 2.3 – Tecnologías de visión artificial y detección de objetos

2.4 Autenticación y Seguridad

Tabla 2.4 – Tecnologías de autenticación y seguridad

2.5 Herramientas de Desarrollo y Control de Versiones

Tabla 2.5 – Herramientas de desarrollo y control de versiones

2.6 Referencias Teóricas y Metodológicas

2.6.1 Cambio Climático y Agricultura

Tabla 2.6.1 – Referencias sobre cambio climático y agricultura

2.6.2 Índices Espectrales Agrícolas

Tabla 2.6.2 – Referencias sobre índices espectrales agrícolas

2.6.3 Machine Learning en Agricultura

Tabla 2.6.3 – Referencias sobre machine learning aplicado a agricultura

2.6.4 APIs REST y Arquitectura de Microservicios

Tabla 2.6.4 – Referencias sobre arquitectura de software y APIs REST

2.7 Documentación Técnica Complementaria

Documentación API agrotech-cultitech

Repositorio GitHub del proyecto

2.1 Fuentes de APIs y Servicios Externos Integrados

Tabla 2.1 – APIs y servicios externos integrados en la plataforma

Servicio	Descripción	Sitio Web	Cita APA
Open-Meteo API	Proporciona datos meteorológicos en tiempo real e históricos para monitorización agrícola.	https://open-meteo.com	Open-Meteo. (2026). Open-Meteo Weather API. Recuperado de https://open-meteo.com
Hugging Face	Plataforma con modelos preentrenados para detección de plagas, identificación de plantas y análisis visual de cultivos.	https://huggingface.co	Hugging Face. (2026). Hugging Face - The AI community building the future. Recuperado de https://huggingface.co
Groq	API de modelos de lenguaje grandes para asistente inteligente agrícola.	https://groq.com	Groq. (2026). Groq API para modelos de lenguaje. Recuperado de https://groq.com
Google Earth Engine	Plataforma de análisis geoespacial para índice NDVI, análisis de suelo y productividad de cultivos.	https://earthengine.google.com	Google Earth Engine. (2026). Google Earth Engine - Planetary-scale geospatial analysis. Recuperado de https://earthengine.google.com

Servicio	Descripción	Sitio Web	Cita APA
AURAVANT (Deprecada)	Plataforma anterior para gestión de parcelas agrícolas.	https://auravant.com	Auravant. (2026). Plataforma de gestión agrícola integrada. Recuperado de https://auravant.com

2.2 Tecnologías y Frameworks - Stack Técnico

FRONTEND

Tabla 2.2.1 – Tecnologías frontend del sistema

Tecnología	Versión	Propósito	Sitio	Cita APA
React	18.x+	Framework para interfaz modular y responsiva	https://react.dev	Facebook Inc. (2026). React - A JavaScript library for building user interfaces. Recuperado de https://react.dev
Vite	5.x	Herramienta de construcción y desarrollo optimizada	https://vitejs.dev	Vite Contributors. (2026). Vite - Next generation frontend tooling. Recuperado de https://vitejs.dev

BACKEND

Tabla 2.2.2 – Tecnologías backend del sistema

Tecnología	Versión	Propósito	Sitio	Cita APA
Node.js	18.x+	Runtime JavaScript para backend escalable	https://nodejs.org	OpenJS Foundation. (2026). Node.js - JavaScript runtime built on Chrome's V8 engine. Recuperado de https://nodejs.org
Express.js	4.x	Framework minimalista para APIs REST	https://expressjs.com	TJ Holowaychuk et al. (2026). Express - Fast, unopinionated, minimalist web framework for Node.js. Recuperado de https://expressjs.com

MICROSERVICIOS DATA SCIENCE

Tabla 2.2.3 – Tecnologías de Data Science e Inteligencia Artificial

Tecnología	Versión	Propósito	Sitio	Cita APA
Python	3.11+	Lenguaje para análisis de datos y modelos de IA	https://python.org	Python Software Foundation. (2026). Python - Programming Language. Recuperado de https://python.org
Flask	2.x	Framework ligero para exponer APIs de servicios de datos	https://flask.palletsprojects.com	Armin Ronacher et al. (2026). Flask - Web development framework for Python. Recuperado de https://flask.palletsprojects.com

Tecnología	Versión	Propósito	Sitio	Cita APA
Scikit-learn	1.3.x	Librería para Machine Learning y predicción de alertas agrícolas	https://scikit-learn.org	Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830. https://jmlr.org/papers/v12/pedregosa11a.html
Pandas	2.x	Manipulación y análisis de datos estructurados	https://pandas.pydata.org	McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 445, 51-56.

PERSISTENCIA DE DATOS

Tabla 2.2.4 – Tecnologías de persistencia y gestión de datos

Tecnología	Versión	Propósito	Sitio	Cita APA
PostgreSQL	14.x+	Base de datos relacional para usuarios, cultivos, predicciones	https://postgresql.org	PostgreSQL Global Development Group. (2026). PostgreSQL - The World's Most Advanced Open Source Relational Database. Recuperado de https://postgresql.org
SQLAlchemy	2.x	ORM para Python - gestión de base de datos	https://sqlalchemy.org	SQLAlchemy Contributors. (2026). SQLAlchemy - The Python SQL Toolkit and Object Relational Mapper. Recuperado de https://sqlalchemy.org

2.3 Visión Artificial y Detección de Objetos

Tabla 2.3 – Tecnologías de visión artificial y detección de objetos

Tecnología	Propósito	Sitio	Cita APA
YOLOv8	Modelo de detección de plagas e identificación de plantas en tiempo real	https://ultralytics.com/yolov8	Ultralytics. (2026). YOLOv8 by Ultralytics - State-of-the-art object detection. Recuperado de https://ultralytics.com/yolov8
OpenCV	Librería de visión artificial para procesamiento de imágenes (v4.x)	https://opencv.org	Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.

2.4 Autenticación y Seguridad

Tabla 2.4 – Tecnologías de autenticación y seguridad

Tecnología	Propósito	Cita APA
JSON Web Tokens (JWT)	Autenticación segura en APIs REST	IETF. (2015). RFC 7519 - JSON Web Token (JWT). Recuperado de https://tools.ietf.org/html/rfc7519

Tecnología	Propósito	Cita APA
Bcrypt	Algoritmo de hash para contraseñas seguras (https://github.com/pyca/bcrypt)	The Python Cryptographic Authority. (2026). Bcrypt - Secure password hashing for Python. Recuperado de https://github.com/pyca/bcrypt

2.5 Herramientas de Desarrollo y Control de Versiones

Tabla 2.5 – Herramientas de desarrollo y control de versiones

Herramienta	Propósito	Sitio	Cita APA
Git / GitHub	Control de versiones y colaboración en equipo	https://github.com	GitHub Inc. (2026). GitHub - Where the world builds software. Recuperado de https://github.com
Visual Studio Code	Editor de código fuente	https://code.visualstudio.com	Microsoft. (2026). Visual Studio Code - Code Editor. Recuperado de https://code.visualstudio.com
Docker	Containerización de servicios (Full Stack y Data Science)	https://docker.com	Docker Inc. (2026). Docker - Containerization Platform. Recuperado de https://docker.com

2.6 Referencias Teóricas y Metodológicas

CAMBIO CLIMÁTICO Y AGRICULTURA

Tabla 2.6.1 – Referencias sobre cambio climático y agricultura

Referencia	Cita APA
IPCC - Panel Intergubernamental sobre el Cambio Climático	IPCC. (2023). Climate Change 2023: Synthesis Report. Recuperado de https://www.ipcc.ch/report/ar6/syr/

ÍNDICES ESPECTRALES AGRÍCOLAS

Tabla 2.6.2 – Referencias sobre índices espectrales agrícolas

Índice	Cita APA
Índice NDVI	Rouse, J. W., Haas, R. H., Schell, J. A., & Deering, D. W. (1974). Monitoring vegetation systems in the Great Plains with ERTS. Third ERTS Symposium, 1, 309-317.
Índices de Humedad de Suelo	Sarmah, S., Das, A. S., & Rajeevan, M. (2020). Soil moisture retrieval using multi-temporal Sentinel-1 data. Remote Sensing of Environment, 245, 111828. https://doi.org/10.1016/j.rse.2020.111828

MACHINE LEARNING EN AGRICULTURA

Tabla 2.6.3 – Referencias sobre machine learning aplicado a agricultura

Tema	Cita APA
Predicción de Plagas con ML	Lobell, D. B., Thau, D., Seifert, C., Engle, E., & Bastiaanssen, W. (2015). A remote sensing approach to estimate global crop production. Nature Food, 4(1), 51-59. https://doi.org/10.1038/s43016-021-00400-w

Tema	Cita APA
Modelos Predictivos de Riesgo Agrícola	Schlenker, W., & Roberts, M. J. (2009). Nonlinear temperature effects indicate severe damages to US crop yields under climate change. <i>Proceedings of the National Academy of Sciences</i> , 106(37), 15594-15598. https://doi.org/10.1073/pnas.0906865106

APIS REST Y ARQUITECTURA MICROSERVICIOS

Tabla 2.6.4 – Referencias sobre arquitectura de software y APIs REST

Tema	Cita APA
Principios de Diseño API REST	Fielding, R. T. (2000). <i>Architectural Styles and the Design of Network-based Software Architectures</i> (Doctoral dissertation). University of California, Irvine.
Arquitectura de Microservicios	Newman, S. (2015). <i>Building Microservices: Designing Fine-Grained Systems</i> . O'Reilly Media.

2.7 Documentación Técnica Complementaria

- Documentación API agrotech-cultitech: Ver Anexo 1
- Repositorio GitHub del proyecto: [Insertar URL del repositorio]